

Evan M Drake

02/03/2025

Contents

1	Function Handles	2
2	Definite Integration	2
3	Derivative Estimation	3
4	Control Flow	4

1 Function Handles

Function syntax is not easily understood by computational models. MATLAB has a way of encoding this syntax using *function handles*. A function handle is declared like any other variable but with the added @ operator. Examples can be seen below

$$f(x) = x^2 \tag{1}$$

```
f = @(x) x.^2
```

See that Eq (1) is equivalent to the above MATLAB code. This works for multi-variable functions as well.

$$g(x, y) = x^2 - y^2 \tag{2}$$

```
g = @(x, y) (x.^2 - y.^2)
```

The above is only the single line way to define a function handle, it is perfectly acceptable to write a full MATLAB as well. The only difference is that you will simply use the name of the MATLAB function.

2 Definite Integration

Integrals in continuous space are defined by equations and can have infinite or uncountable solutions. Definite integrals exist in discrete space and thus can be approximated to some value, this is regularly called *integral approximation*. There are a multitude of methods to this including the Riemann sum method(s), trapezoid method, and Simpson's rule. Here we will learn the trapezoid method ¹ as it seems most relevant to engineers. The intuition of the trapezoid method is to imagine a bunch of trapezoids filling the integration space, then simply adding their respective areas together. The method implies regular trapezoids but accounts for non-regularity (it is worth noting, all computers force regularity). The trapezoid rule follows from the area of a trapezoid.

¹though I plan to give example of Riemann's method in class.

$$\frac{h}{2} * (a + b) \quad (3)$$

The area of a trapezoid can be seen in Eq (3) where h is the height, and a, b are base lengths. Thus we can use this to estimate some integral in the following style.

$$\frac{h * (f(x_a) + f(x_b))}{2} \quad (4)$$

In Eq (4) we see similarity with the area of a trapezoid but instead of side lengths a, b we have $f(x_a), f(x_b)$. This is due to us using points of the fn here. We estimate the function by utilizing the perpendicular axis and the fn f to draw out trapezoid bases. Thus we can derive the height (or subinterval length) to be $h = |x_a - x_b|$. Thus we can chain this and create as many trapezoids as we may where the height now becomes (commonly) $\Delta x = |x_k - x_{k+1}|$.

$$\sum_{k=1}^N f(x_{k+1}) + f(x_k) \frac{\Delta x}{2} \quad (5)$$

MATLAB defines the built-in function *int* to define both definite integrals and continuous.

```
f = @(x) x.^2
int(<function handle>, <lower bound>, <upper bound>)
int(f, 0, 10)
```

3 Derivative Estimation

We can intuit the derivative metric as estimating the tangential slope in discrete space. This indicates that a naive estimation is to take the slope of a secant line at some near by points x_a and x_b .

$$\frac{f(x_a) - f(x_b)}{h} \quad (6)$$

$$h = |a - b|$$

The above expression is known as Newton's difference quotient. The slope of a secant line between some x_a and x_b differs from the slope of the tangent line by approx a value of h . Thus the *exact* derivative is something like

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x_b) - f(x_a)}{h} \quad (7)$$

```
x = [1,2,3,4,5,6,.....]
y = [2,3,4,5,6,7,8,.....]

diff(<list of data points>)./diff(<list of data points>)

diff(x)./diff(y)
```

4 Control Flow

Thus far in our programmatic patterns we have only explored sequential or linear styles.

$$A \rightarrow B \rightarrow C$$

These styles portray a linear thought pattern, However almost no problem has a truly linear solution. For instance if we think about just everyday events:

- If it is raining then I will take an umbrella
- If it is cold then I will light a fire
- If my teacher goes on another tangent in class then I will take a nap

Most problems are systems of cause an effect. In something like a programming language, MATLAB for instance we have a special structure to describe these systems: *if-then-else*. An *if-then-else* structure allows us to take account of all probabilistic outcomes. We take advantage of conditional statements *Booleans*, using them to track significant changes in logic.

```
if x == 5
    A(r,c) = 5
```

```
elseif x > 5
    A(r,c) = 10
else
    A(r,c) = 1
```

We can see in the above code that we utilize Booleans (true or false statements) to fork the logic. What does this look like more generally?

```
if <boolean expression>
    <logic>
<optional elseif expression for more cases>
else <boolean expression>
    <logic>
```